

Машинное обучение, ФКН ВШЭ

Практическое задание 6. Бустинговое

Общая информация

Дата выдачи: 11.05.2026

Мягкий дедлайн: 25.05.2026 23:59 MSK

Жёсткий дедлайн: 31.05.2026 23:59 MSK

Оценивание и штрафы

Каждая из задач имеет определенную «стоимость» (указана в скобках около задачи). Максимально допустимая оценка за работу — 10 баллов.

Задание выполняется самостоятельно. «Похожие» решения считаются плагиатом и все задействованные студенты (в том числе те, у кого списали) не могут получить за него больше 0 баллов (подробнее о плагиате см. на странице курса). Если вы нашли решение какого-то из заданий (или его часть) в открытом источнике, необходимо указать ссылку на этот источник в отдельном блоке в конце вашей работы (скорее всего вы будете не единственным, кто это нашел, поэтому чтобы исключить подозрение в плагиате, необходима ссылка на источник).

Неэффективная реализация кода может негативно отразиться на оценке.

Формат сдачи

Задания сдаются через систему anytask. Посылка должна содержать:

- Ноутбук homework-practice-06-Username.ipynb
- Модуль boosting.py

Username — ваша фамилия на латинице

О задании

В этом задании вам предстоит вручную запрограммировать один из самых мощных алгоритмов машинного обучения — бустинг. Однако с большой силой приходит и большая ответственность, поэтому придется научиться им пользоваться

```
In [1]: from warnings import filterwarnings
```

```
import numpy as np
import pandas as pd
import seaborn as sns
from sklearn.model_selection import train_test_split

sns.set(style='darkgrid')
filterwarnings('ignore')
```

Мы будем работать с датасетом по предсказанию гендера. Никакие фичи мы придумывать не будем, но если вам интересно, в датасете имеется:

- region — регион, в котором проводили опрос
- age — возраст опрошенного
- year — год опроса
- race — этническая принадлежность
- sch — количество классов образования
- empstat — статус трудоустройства
- occ — код профессии
- ind — код индустрии, в которой человек работает
- annhrs — число часов работы в год
- hrwage — почасовая оплата
- icwage — суммарная оплата в год
- occupation — должность

Целевая метрика ROC-AUC, потому что мы непритязательны и это не фокус этой домашки

```
In [2]: df = pd.read_parquet("hw6_dataset.pq")

X = df.drop("sex", axis=1)
y = df["sex"]
```

Пока что мы временно выкинем колонку с должностью, но мы к ней обязательно вернемся

Разделим на обучающую, валидационную и тестовую выборки по годам, чтобы предсказывать будущее, а не прошлое

```
In [3]: X_train = X[X["year"] < 2011].drop("occupation", axis=1).values
y_train = y[X["year"] < 2011].values

X_valid = X[X["year"] == 2011].drop("occupation", axis=1).values
y_valid = y[X["year"] == 2011].values

X_test = X[X["year"] == 2013].drop("occupation", axis=1).values
y_test = y[X["year"] == 2013].values

X_train.shape, X_valid.shape, X_test.shape
```

```
Out[3]: ((236640, 10), (53857, 10), (53790, 10))
```

Домашнее задание имеет нелинейную структуру. В нём есть  **обязательные** части, без которых будет тяжело **(6 баллов)**, и целая куча  **опциональных**,

которыми можно добить оценку до 10, делайте то, что нравится больше!
Музыкальный бонус идёт отдельно, за него можно получить ещё 0.5 балла.

Задание 1. Базовый градиентный бустинг (4 балла)

Это задание  **обязательное**, без него домашка не имеет смысла! Ещё раз обращаем внимание, что **опциональные** задания помечены 

Первая часть посвящена реализации собственного градиентного бустинга. Обращаем внимание, что пользоваться готовыми реализациями **нельзя**, если специально не оговорено. На все **вопросы** должен быть **ответ**. В заданиях, где нужно делать параметры, часто есть референсы из других библиотек. На случай, если объяснение из ноутбука непонятно, советуем обратиться к соответствующей документации и пощупать, как это делается там

Вам нужно дописать код в файлике `boosting.py`. Для вас уже подготовлен шаблон класса `Boosting`, вы можете менять его по своему усмотрению.

Инструкции для функций:

`__init__`

Существующие параметры:

- `base_model_class` — класс базовой модели нашего бустинга
- `base_model_params` — словарь с гиперпараметрами для базовой модели
- `n_estimators` — какое количество базовых моделей нужно обучить
- `learning_rate` — темп обучения, должен быть из полуинтервала $(0, 1]$
- `random_state` — в роли рандом стейта
- `verbose` — для тех, кому не нравится загадочность, можно включить и увидеть, сколько еще ждать конца обучения

`fit`

В `fit` приходит выборка, на которой мы обучаем новые базовые модели

1. Сначала нам нужно сделать какую-то нулевую модель, сделать предсказания (в шаблоне это нулевая модель, соответственно предсказания это просто `np.zeros`).
2. После этого нужно обучить `n_estimators` базовых моделей (как и на что обучаются базовые модели смотрите в лекциях и семинарах).
3. После каждой обученной базовой модели мы должны обновить текущие предсказания, посчитать ошибку на выборке (используем `loss_fn` для этого) и найти новую оптимальную гамму

partial_fit

В `partial_fit` передаются данные `X: np.ndarray, y: np.ndarray` для одного шага бустинга: обучаем следующую базовую модель на антиградиенте и обновляем ансамбль.

predict_proba

В `predict_proba` приходит любая выборка, нужно предсказать вероятности для неё. Суммируем предсказания базовых моделей на этой выборке (не забываем про гаммы) и накидываем сигмоиду

```
In [4]: %load_ext autoreload
```

```
In [5]: %autoreload 2
```

```
from boosting import BoostingClassifier
```

Проверка кода

```
In [6]: boosting = BoostingClassifier(
    verbose=True,
    base_model_params=dict(max_depth=5),
    n_estimators=50,
    random_state=69, # не забывайте фиксировать, пожалуйста
)

boosting.fit(X_train, y_train)

assert len(boosting.models) == boosting.n_estimators
assert len(boosting.gammas) == boosting.n_estimators

assert boosting.predict_proba(X_test).shape == (X_test.shape[0], 2), "prediction
assert np.array_equal(boosting.history["train_loss"], np.sort(boosting.history["

assert boosting.predict_proba(X_test).shape == (X_test.shape[0], 2)

# здесь и далее работаем с метрикой ROC-AUC
print(f'Train ROC-AUC {boosting.score(X_train, y_train):.4f}')
print(f'Valid ROC-AUC {boosting.score(X_valid, y_valid):.4f}')
print(f'Test ROC-AUC {boosting.score(X_test, y_test):.4f}')

0%|          | 0/50 [00:00<?, ?it/s]
Train ROC-AUC 0.8013
Valid ROC-AUC 0.7247
Test ROC-AUC 0.7196
```

Задание 2. Имплементация гиперпараметров (🚩 1 балл | 💎 8.25 балла)

Бустинг, как метод агрегации, предлагает несколько вариантов регуляризации.

В этом пункте мы будем делать кучу самых разных гиперпараметров, каждый из них прекрасен по-своему, но полноценно их эффект получится ощутить лишь на этапе их перебора (кроме, пожалуй, `rsm`), поэтому делать третью часть до второй не рекомендуется

Важно! После добавления каждого параметра проверьте, как это повлияет на качество. Специально крутить параметры не нужно, но важно увидеть, что перформанс модели изменился, в худшую или в лучшую сторону. Для этого какие-то параметры нужны, пусть и от балды, **гарантии, что дефолтные параметры подойдут, нет**. Лучше всего это видно по графикам обучения **на трейне и валидации**, рекомендуется смотреть именно на них

🚧 2.1. Борьба с переобучением (0.5 балла)

Как известно, бустинги бывают склонны к переобучению. Чтобы этого не допустить, можно контролировать процесс обучения и проверять критерий останова на валидации прямо во время обучения. Реализуйте такую процедуру, не забудьте, что всю предобработку, что вы будете в дальнейшем делать для трейна, нужно будет повторить на валидации

Добавьте в вашу модель:

- `early_stopping_rounds: int | None = 0` — число раундов для критерия останова. Если качество ухудшается на протяжении всех раундов подряд, останавливаем обучение
- `eval_metric: str | None = None` — метрика, на которую смотрим, чтобы остановить обучение, может быть ключом из `model.history`
- `plot_history(self, keys: str | Iterable[str])` — метод, который строит график изменения метрики по ключам из `model.history`

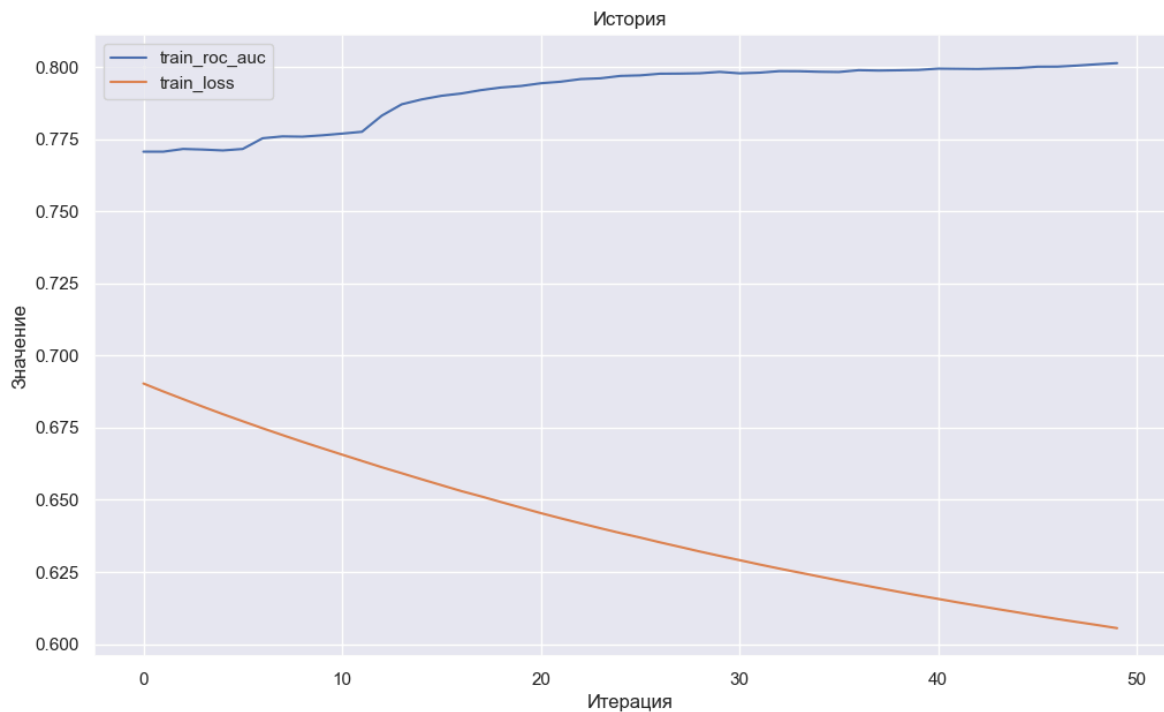
И конкретно в метод `model.fit`:

- `eval_set: tuple[np.ndarray] | None = None` — валидационная выборка `X_valid: np.ndarray, y_valid: np.ndarray`, на которой будем проверять переобучение
- `use_best_model: bool = False` — использовать лучшую модель, или нет. Если да, то отрезаем гаммы и модели вплоть до той, на которой остановились, минус `early_stopping_rounds`, т.е самой лучшей, после которой качество только падает

XGBoost **LightGBM** **CatBoost**
`early_stopping_rounds`, `eval_set`

Продемонстрируйте, что все работает

```
In [7]: boosting.plot_history(['train_roc_auc', 'valid_roc_auc', 'train_loss', 'valid_lo
```



```
In [8]: boosting_early = BoostingClassifier(
    verbose=True,
    base_model_params=dict(max_depth=5),
    n_estimators=50,
    random_state=69,
    eval_metric='valid_roc_auc',
    early_stopping_rounds=5
)

boosting_early.fit(
    X_train, y_train,
    eval_set=(X_valid, y_valid)
)

print(f"Фактически обучено моделей: {boosting_early._n_fitted} из {boosting_early.n_estimators}")

assert boosting_early._n_fitted < boosting_early.n_estimators
assert len(boosting_early.models) == boosting_early.n_estimators

print(f"Train ROC-AUC: {boosting_early.score(X_train, y_train):.4f}")
print(f"Valid ROC-AUC: {boosting_early.score(X_valid, y_valid):.4f}")
```

0%| | 0/50 [00:00<?, ?it/s]

Фактически обучено моделей: 6 из 50

Train ROC-AUC: 0.7715

Valid ROC-AUC: 0.7138

```
In [9]: # *??(*??*?/?*?

boosting_new = BoostingClassifier(
    verbose=True,
    base_model_params=dict(max_depth=5),
    n_estimators=50,
    random_state=69,
    eval_metric='valid_roc_auc'
)

boosting_new.fit(
```

```

X_train, y_train,
eval_set=(X_valid, y_valid),
use_best_model=True
)

print(f"Фактически обучено моделей: {boosting_new.n_fitted} из {boosting_new.n_

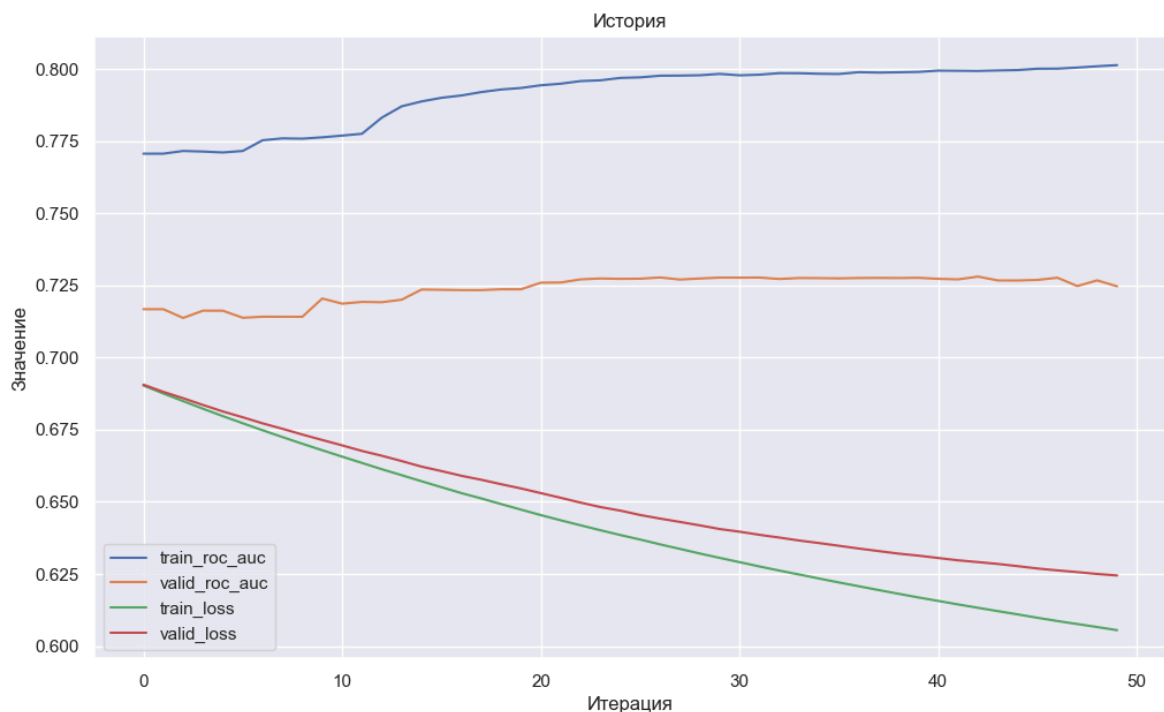
assert boosting_new.predict_proba(X_test).shape == (X_test.shape[0], 2), "predic
assert np.array_equal(boosting_new.history["train_loss"], np.sort(boosting_new.h

print(f'Train ROC-AUC {boosting_new.score(X_train, y_train):.4f}')
print(f'Valid ROC-AUC {boosting_new.score(X_valid, y_valid):.4f}')
print(f'Test ROC-AUC {boosting_new.score(X_test, y_test):.4f}')

boosting_new.plot_history(['train_roc_auc', 'valid_roc_auc', 'train_loss', 'vali

```

0% | 0/50 [00:00<?, ?it/s]
Фактически обучено моделей: 50 из 50
Train ROC-AUC 0.8013
Valid ROC-AUC 0.7247
Test ROC-AUC 0.7196



📌 2.2. Категории 2.0 (0.5 балла)

Пока что для простоты картины мы игнорировали категориальные признаки.

CatBoost в частности предлагает один из самых эффективных и быстрых вариантов кодировки категорий, который мы попробуем повторить

Основная идея в том, что важны не сами категории, а значения целевой переменной, которые имеют объекты этой категории. Каждый категориальный признак мы заменим средним значением целевой переменной по всем объектам этой же категории:

$$g_j(x, X) = \frac{\sum_{i=1}^{\ell} [f_j(x) = f_j(x_i)] [y_i = +1]}{\sum_{i=1}^{\ell} [f_j(x) = f_j(x_i)]}$$


```

)

encoder = LabelEncoder()
X_train_label = X_train.copy()
X_train_label[:, -1] = encoder.fit_transform(X_train[:, -1])

X_valid_label = X_valid.copy()
X_valid_label[:, -1] = encoder.transform(X_valid[:, -1])

X_test_label = X_test.copy()
X_test_label[:, -1] = encoder.transform(X_test[:, -1])

boosting_no_cat.fit(X_train_label, y_train, eval_set=(X_valid_label, y_valid))

print('Без кодирования категорий:')
print(f'Train ROC-AUC {boosting_no_cat.score(X_train_label, y_train):.4f}')
print(f'Valid ROC-AUC {boosting_no_cat.score(X_valid_label, y_valid):.4f}')
print(f'Test ROC-AUC {boosting_no_cat.score(X_test_label, y_test):.4f}')

boosting_cat.plot_history(['train_roc_auc', 'valid_roc_auc', 'train_loss', 'valid_loss'])
boosting_no_cat.plot_history(['train_roc_auc', 'valid_roc_auc', 'train_loss', 'valid_loss'])

```

С кодированием категорий:

Train ROC-AUC = 0.9557

Valid ROC-AUC = 0.7982

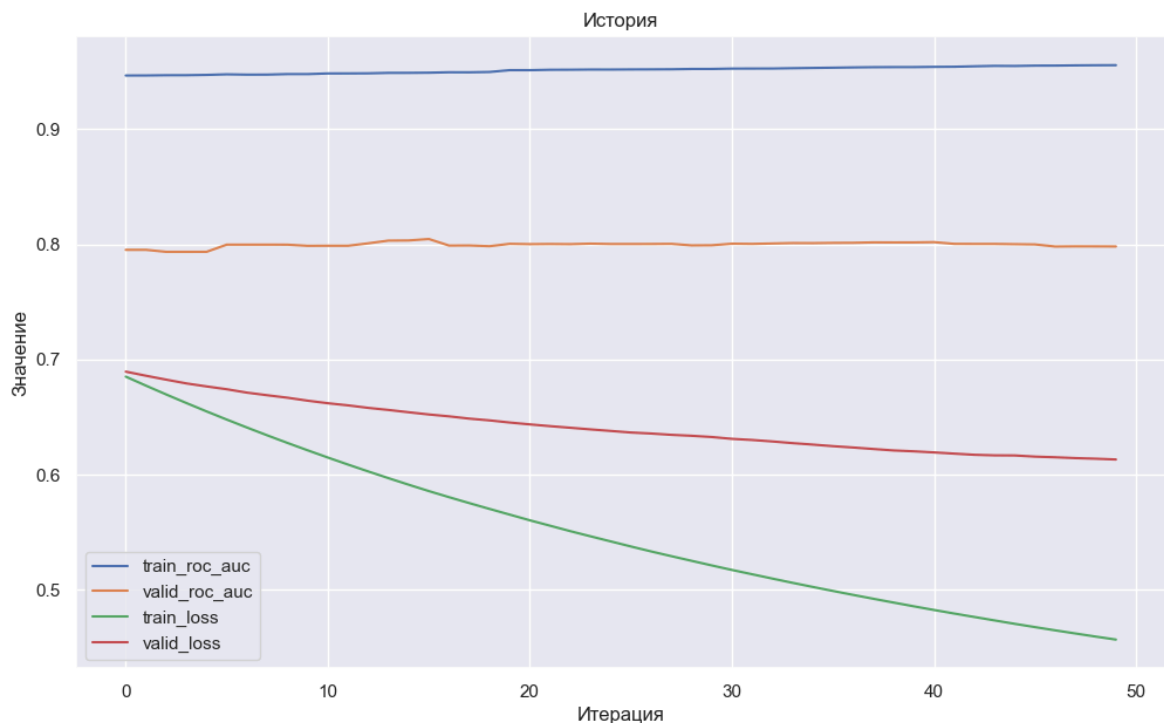
Test ROC-AUC = 0.7913

Без кодирования категорий:

Train ROC-AUC 0.8234

Valid ROC-AUC 0.7537

Test ROC-AUC 0.7455




```
boosting_ordered.plot_history(['valid_roc_auc'])  
boosting_no_cat.plot_history(['valid_roc_auc'])
```

Train ROC-AUC = 0.9557

Valid ROC-AUC = 0.7982

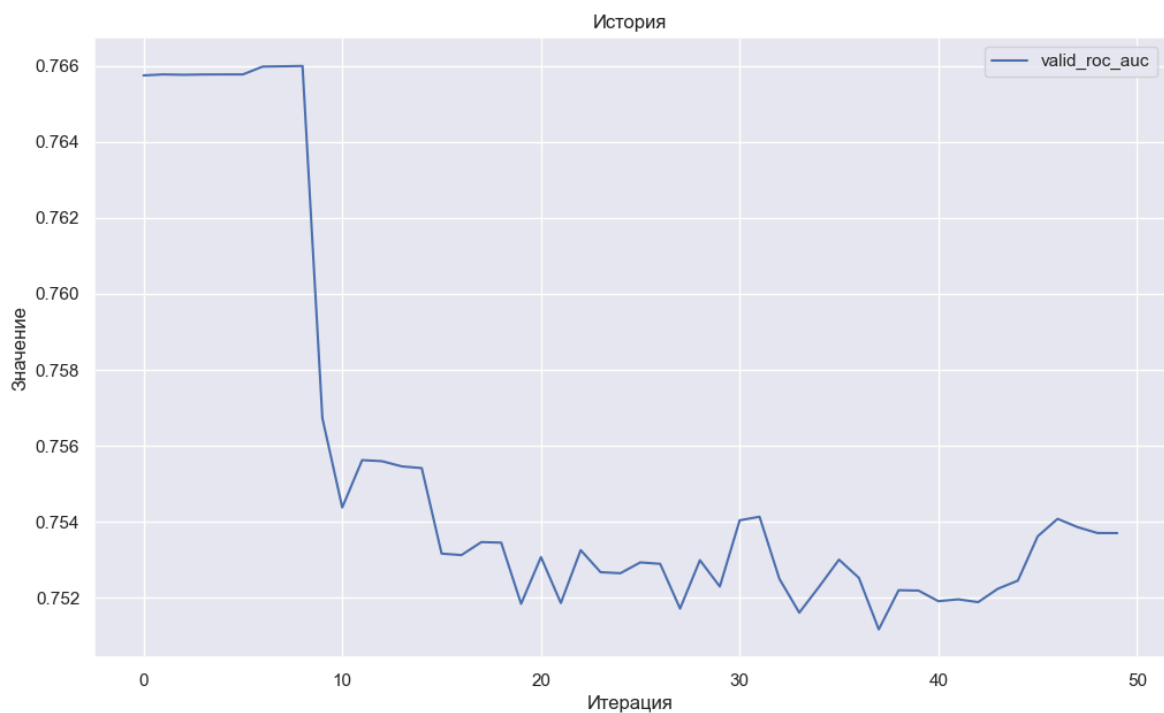
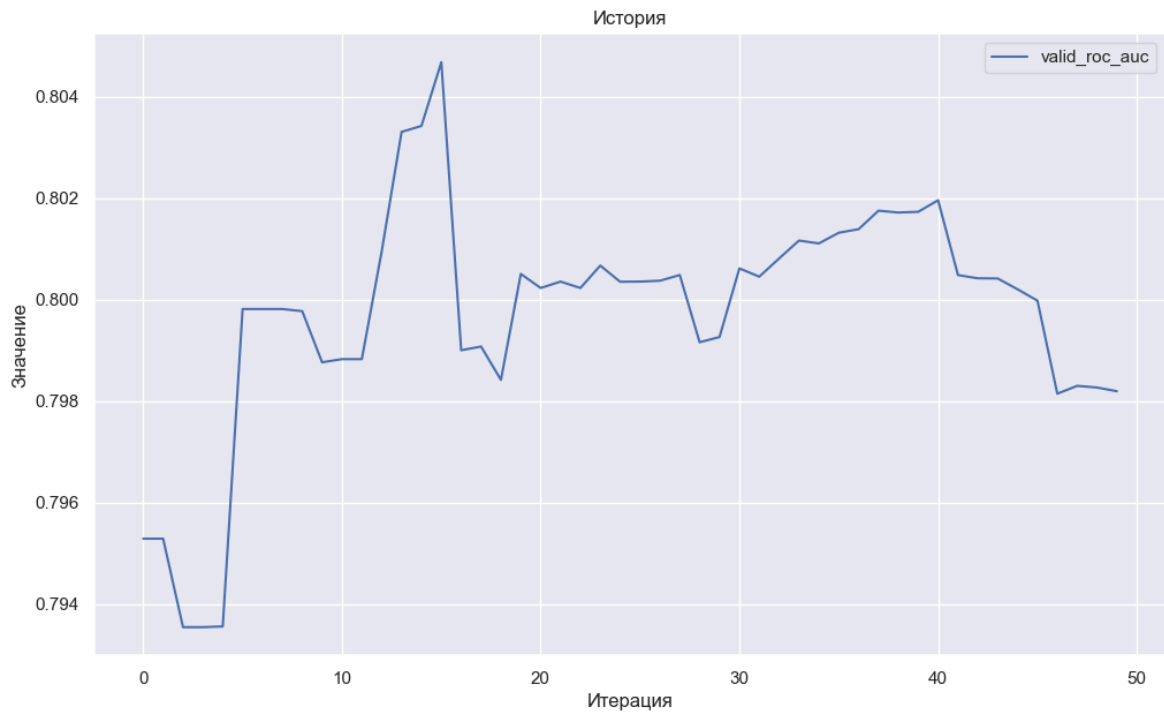
Test ROC-AUC = 0.7913

Без кодирования категорий:

Train ROC-AUC 0.8234

Valid ROC-AUC 0.7537

Test ROC-AUC 0.7455



2.4. Категории 4.0, L2 и боль (2 балла)

Но погодите, вы ещё ничего не видели! Счётчики это замечательный способ кодирования категорий, но у них есть ровно один недостаток — после кодировки

признаки становятся числовыми, а если вы помните дз5, категориальные и числовые признаки обрабатываются деревьями чуть-чуть иначе.

LightGBM и **XGBoost** решают этот вопрос не так, как **CatBoost**. Сейчас будем разбираться... И будет это тяжело, потому что отдебажить это не получится вплоть до самого конца.

Для этого пункта вам придётся стряхнуть пыль с вашей реализации дерева из дз5. Будем верить, что она написана красиво и хорошо масштабируется на новые МЛные приключения.

2.4.1. Настоящий критерий информативности (0.5 балла)

Современные реализации бустинга не используют дефолтные критерии информативности, вроде энтропии или дисперсии. Параметр `gamma`, который за нас любезно подбирается в классе `Boosting` это немного устаревший способ выбора весов (лет так на 25).

На лекции обсуждается, что получить формулу для обучения нового дерева $b_n(x)$ можно, если разложить исходную задачу обучения бустинга в ряд Тейлора. Там же показано, что при обучении на один лишь сдвиг $s_i(x)$ мы полагаем кривизну $h_i(x) = \frac{\partial^2 L}{\partial z^2}$ константной. Но можем и не полагать, как **сделали** авторы уже немолодого **XGBoost**.

Критерий информативности в таком случае имеет вид:

$$H(R_m) = \frac{G_l^2}{H_l + \lambda} + \frac{G_r^2}{H_r + \lambda} - \frac{G_m^2}{H_m + \lambda}$$

- G — сумма градиентов
- H — сумма гессианов
- λ — коэффициент регуляризации
- индексы l, r, m — левый, правый, исходный узел дерева соответственно

Кривизна позволяет нам подстроиться под каждый объект, когда мы делаем шаг градиентного бустинга. γ в таком случае оказывается не нужна, она не умеет подстраивать шаг индивидуально. Более того, сюда можно сразу же встроить L_2 регуляризацию, что мы и сделаем. Наконец, если перейти от объектов к градиентам и гессианам, внезапно можно вообще забыть на то, числовой признак или категориальный, мы же всё равно оцениваем градиент, то есть сплит подбираем напрямую из задачи. Велико!

Текущая задача: модифицировать функцию `find_best_split`. Считайте, что параметры `grad_vector` и `hess_vector` в неё уже передаются. Градиент мы считать уже умеем, с гессианом разберёмся попозже

2.4.2. Я посадил дерево (0.75 балла)

Теперь, когда критерий у нас есть, наступает следующий логический шаг - пихнуть его в дерево. Предсказания будут иметь следующий вид:

$$b(R) = \frac{1}{|R|} \sum_{y_i \in R} \frac{\nabla(y_i)}{(\nabla^2(y_i) + \lambda)}$$

Это сумма градиентов делить на сумму гессианов + коэффициент регуляризации. В случае $\nabla^2(y_i) = 1$ и нулевой регуляризации вырождается в наш самый обыкновенный бустинг. Мы ведь помним, что каждое дерево в нём обучается на антиградиент.

Что делать то?

- добавить параметр `l2: float | None = 1.0` в вашу реализацию дерева
- подправить предсказания по формуле выше, сейчас они считаются для классификации, а не для регрессии (должно быть идентично и для чисел, и для категорий)
- категории нужно занумеровать, это делается за вас в дереве, но не забудьте пожалуйста
- в `fit_node` должны поступать градиенты и гессианы, в левый и правый узел после разбиения тоже, уже для соответствующих левой и правой подвыборок

Строго говоря, **LightGBM** перебирает категории по критерию Фишера, который группирует категории по градиентам. Это один из видов квантизации, которая будет чуть ниже. Делать это не надо, но если любопытно, то [почитайте](#)

```
In [ ]: # *??*??*??*??*??*??
```

2.4.3. Момент истины (0.75 балла)

Наконец, осталось запихнуть это дело в бустинг. Алгоритм будет работать точно так же. Если вы корректно его написали, поменять потребуется немного, а именно:

- добавить гиперпараметр `l2: float | None = 0.0`
- добавить метод `self.hess_fn = lambda y, z: ...`, который будет считать гессиан для бинарной кросс-энтропии
- приравнять оптимальную гамму к единице, если `l2` не равен 0
- передавать гредиенты и гессианы в базовую модель, если у неё есть параметр `l2` и он не равен 0
- гессиан лучше клипать, иначе можно внезапно поделить на 0 при маленьком `l2`
- не забудьте, что передавать надо антиградиенты, а не просто градиенты, иначе выйдет что-то не то

Покажите, что оно работает...

2.5. Бутстрап (1.25 балла)

Как известно, при обучении базовых алгоритмов в беггинге, используется бутстрапированная выборка. Звучит хайпово, почему бы не попробовать сделать то же самое в бустинге?

Идея, которая за этим стоит проста — бустинги переобучаются, как черти. Бутстрап же может сделать процесс куда более робастным и быть одним из способов регуляризации, а заодно уменьшить время обучения, в случае бутстрапа Бернулли

2.5.1. Бутстрап дефолтный (0.75 балла)

Вам нужно реализовать параметры:

- `subsample: float = 1.0` — доля объектов, на которой будет обучаться базовая модель (какую часть составляет бутстрапная выборка от исходной обучающей). Соответствует бутстрапу типа `Bernoulli`. Каждый объект либо входит в подвыборку, либо нет
- `bagging_temperature: float = 1.0` — температура для генерации весов. Соответствует бутстрапу типа `Bayesian`. Каждому объекту присваивается вес w , который получается из температуры t , после чего объект, в количестве w , попадает в выборку.

$$w = (-\log(X))^t, \text{ где } X \sim U[0, 1]$$

Таким образом можно сделать симуляцию повторений, обобщив на непрерывный случай. Обычный бутстрап не умеет сэмплировать дробное число объектов, но можно сделать примерно то же самое, если размножить объект w раз, либо присвоить вес. Подумайте, как это корректно реализовать в бустинге

- `bootstrap_type: str | None = 'Bernoulli'` — тип бутстрапа

XGBoost

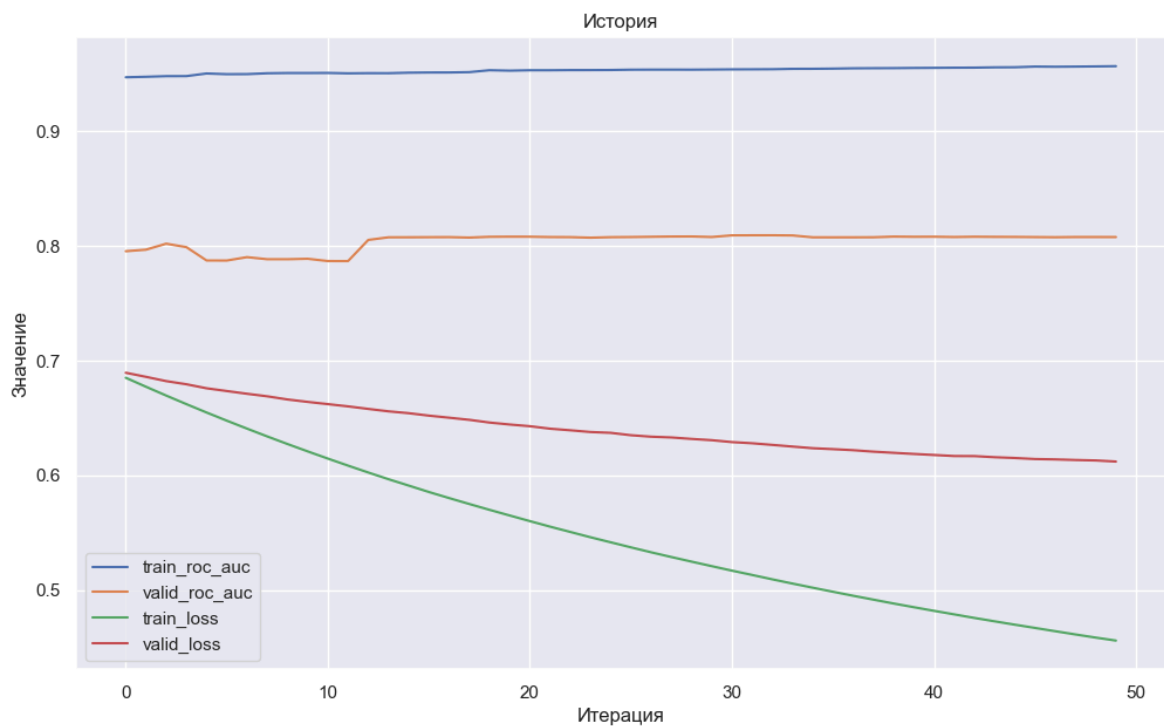
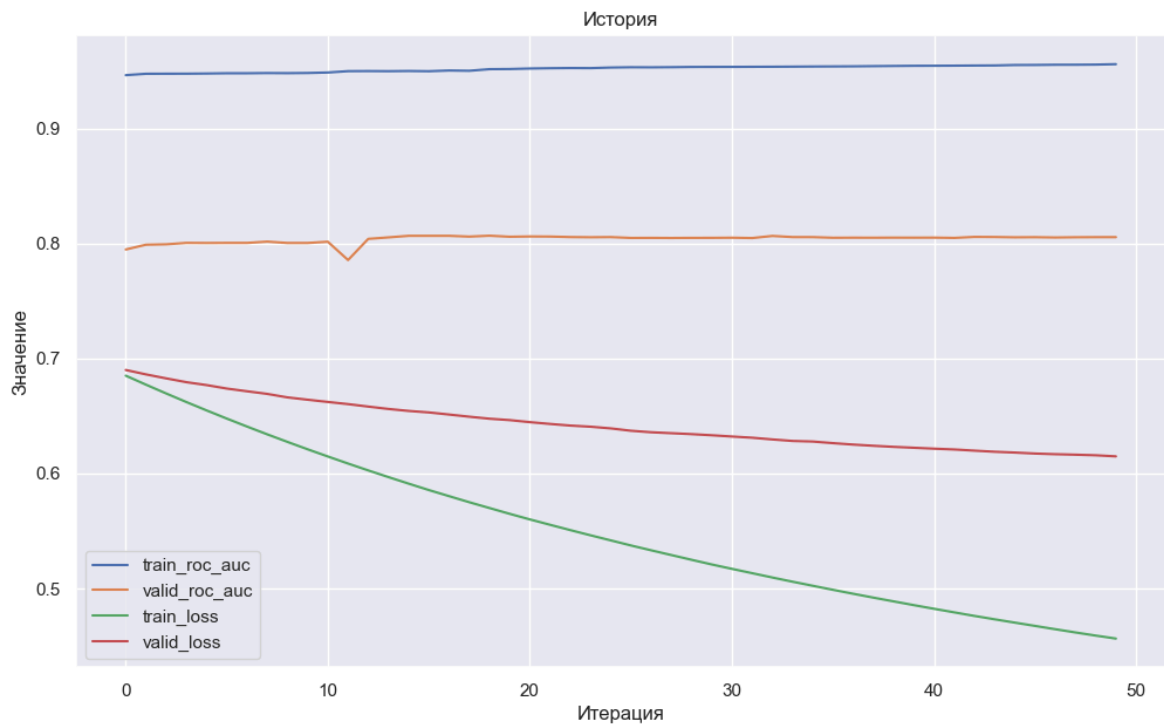
LightGBM

CatBoost

`subsample` , - `subsample` , `bagging` `subsample` , `bootstrap_type`

Вопрос: как бутстрап может помочь в обучении с точки зрения смещения, разброса и вообще?

Ответ: Разброс обычно следует из-за переобучения и плохой обобщающей способности, бутстрап должен помочь решить проблему переобучения, а значит и снизить разброс. Дополнительно это помогает сделать каждую следующую модель менее похожую, коррелирующую с предыдущей за счет разных подвыборок, это помогает улучшить результаты и снизить переобучение тоже. На смещение это не должно



Помимо выбора случайных объектов, есть идея выбирать случайное подмножество признаков, заложенная в Random Forest. Мотивация это делать точно та же самая, что и в бутстрапировании, за это будет отвечать параметр `rsm`

В библиотеках это реализовано так:

XGBoost	LightGBM	CatBoost
<code>colsample_bytree</code>	<code>colsample_bytree</code>	<code>rsm</code>

Нужно добавить параметр:

- `rsm: float = 1.0` — доля признаков, участвующих в обучении модели. Каждый признак либо входит, либо не входит в пул, на котором обучается


```

eval_metric='valid_roc_auc',
cat_features=cat_features,
goss=True,
goss_k=0.2,
subsample=0.3
)
boosting_goss.fit(X_train, y_train, eval_set=(X_valid, y_valid))
print(f'Train ROC-AUC = {boosting_goss.score(X_train, y_train):.4f}')
print(f'Valid ROC-AUC = {boosting_goss.score(X_valid, y_valid):.4f}')
print(f'Test ROC-AUC = {boosting_goss.score(X_test, y_test):.4f}')

```

Train ROC-AUC = 0.9572

Valid ROC-AUC = 0.8049

Test ROC-AUC = 0.7978

2.6. Бинаризация (1 балл)

Вместо того, чтобы выкидывать признаки, можно попробовать проредить их множество значений, применив бинаризацию. В идеале это стоит делать с датасетом до обучения модели, но не обязательно, можно, например, на этапе `fit`, а в `partial_fit` приходит уже квантизованный датасет.

Зачем это? Да все очень просто! Квантизованные данные весят гораздо меньше, значений в них куда как меньше, а значит, наше обучение пойдет гораздо быстрее, к тому же все снова станет устойчивее.

Беда лишь в новых данных, которые могут оказаться вне бина, но великие силы избавили нас от этих проблем в нашем датасете.

Фактически мотивация вновь та же, что у бутстрапа

2.6.1. Простейшие стратегии (0.5 балла)

Нужно добавить параметры:

- `quantization_type: str | None = None` — тип квантизации, если она есть
- `nbins: int = 255` — число бинов для квантизации. Игнорируем, если тип квантизации не указан

Либо сделать квантизатор отдельным классом, где в `fit` запоминаются пороги, а в `transform` преобразуются фичи

Рассмотрим варианта `quantization_type`:

- `'uniform'` — значения признака $[\min f_i, \max f_i]$ отображаются в `nbins` отрезков одинаковой длины, которые полностью покрывают интервал
- `'quantile'` — тот же самый отрезок сортируется и бьется на `nbins` квантилей от 0 до 1

XGBoost

LightGBM

CatBoost

`tree_method`

`max_bin`

`quantize(...)`

Разницу между ними вы можете увидеть на картинке, где квантизуется гауссиана

No description has been provided for this image

Реализуйте их, примените их к числовым признакам, и покажите, что все работает четко

In []: # *??*??*??*??*??

Вопрос: как квантизация поможет в обработке выбросов и нанов?

Ответ:

Вопрос: можно ли просто заполнить наны каким-то числом? Имеет ли значение, что это за число (-1, -100, -1e32)?

Ответ:

2.6.2. Квантизация (0.5 балла)

В современных имплементациях бустинга используются более сложные методы квантизации. К сожалению, способы выше страдают либо от неравноправности бинов — в каких-то объектов больше, в каких-то меньше, в каких-то значения повторяются, в каких-то нет, либо от того, что целевая переменная от бинов зависит опосредованно, да и пороги берутся с потолка

Есть два необычных, но крайне интересных подхода:

1. **MinEntropy**. Это [один из методов](#), который используется для бинаризации в **CatBoost**, остальные на него очень похожи. Будем жадно набирать бины так, чтобы суммарная побиновая энтропия объектов ($\sum_{i \in \text{bin}} x_i \log(x_i)$) была как можно меньше. Это, правда, предполагает, что значения числовых фичей могут повторяться, и так и есть в части наших колонок
2. **PiecewiseEncoding**. Это относительно свежий метод для [табличных трансформеров](#). Суть заключается в том, что бины для квантизации будем брать не с потолка, а именно те, что нужны для разбиения таргета через максимизацию **InfoGain**

Оба метода на самом деле похожи, поэтому выбирайте на свой вкус. Оба пытаются подобрать пороги при помощи некоторого алгоритма, который тоже работает с энтропией или **InfoGain**, возможно, вы его даже знаете, осталось правильно его применить. В библиотеках способы реализации чуть отличаются, и предлагаемый способ не единственный. Если есть желание - экспериментируйте!

```
- - feature_border_type="Piecwise"
```

Добавьте одну из опций, 'min_entropy' или 'piecwise', для quantization_type и покажите, что оно работает

```
In [ ]: # *??*??*?/?*?
```

❖ 2.7. Важность дерева (0.5 балла)

Поскольку наша базовая модель — это дерево из sklearn, мы можем вычислить важность признака отдельно для каждого дерева и взвешенно усреднить (воспользуйтесь feature_importances_ у DecisionTreeRegressor и знанием о гаммах), после этого нормировать значения, чтобы они суммировались в единицу (обратите внимание, что они должны быть неотрицательными — иначе вы что-то сделали не так).

Как правило, этот способ учитывает, сколько раз та или иная фича использовалась для сплита. Чем больше раз, тем этот признак важнее

Допишите к бустингу:

- get_feature_importance(self, X: np.ndarray | None = None, y: np.ndarray | None = None, type="split") — атрибут, который возвращает важность признаков, согласно методу выше

XGBoost

LightGBM

CatBoost

```
model.get_score()
```

```
lightgbm.importance()
```

```
model.get_feature_importance()
```

Покажите, какие признаки оказались самыми важными

```
In [15]: # *??*??*?/?*?
import matplotlib.pyplot as plt

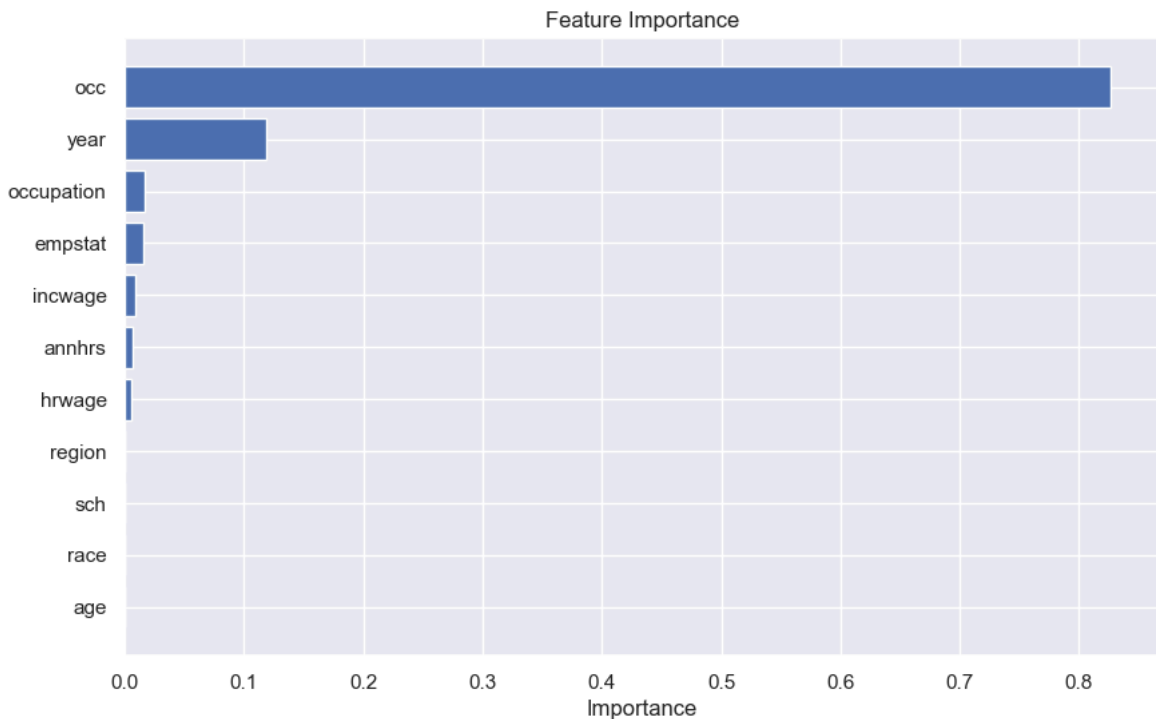
importance = boosting_ordered.get_feature_importance(type='split')
feature_names = df.drop("sex", axis=1).columns.tolist()

order = np.argsort(importance)[::-1]

print("Важность признаков:")
for i in order:
    print(f"{feature_names[i]}: {importance[i]:.4f}")

plt.figure(figsize=(10, 6))
plt.barh(range(len(importance)), importance[order], align='center')
plt.yticks(range(len(importance)), [feature_names[i] for i in order])
plt.xlabel('Importance')
plt.title('Feature Importance')
plt.gca().invert_yaxis()
plt.show()
```

Важность признаков:
occ: 0.8264
year: 0.1183
occupation: 0.0168
empstat: 0.0160
incwage: 0.0092
annhrs: 0.0071
hrwage: 0.0061
region: 0.0000
sch: 0.0000
race: 0.0000
age: 0.0000



2.8. Важность бустинга (1 балл)

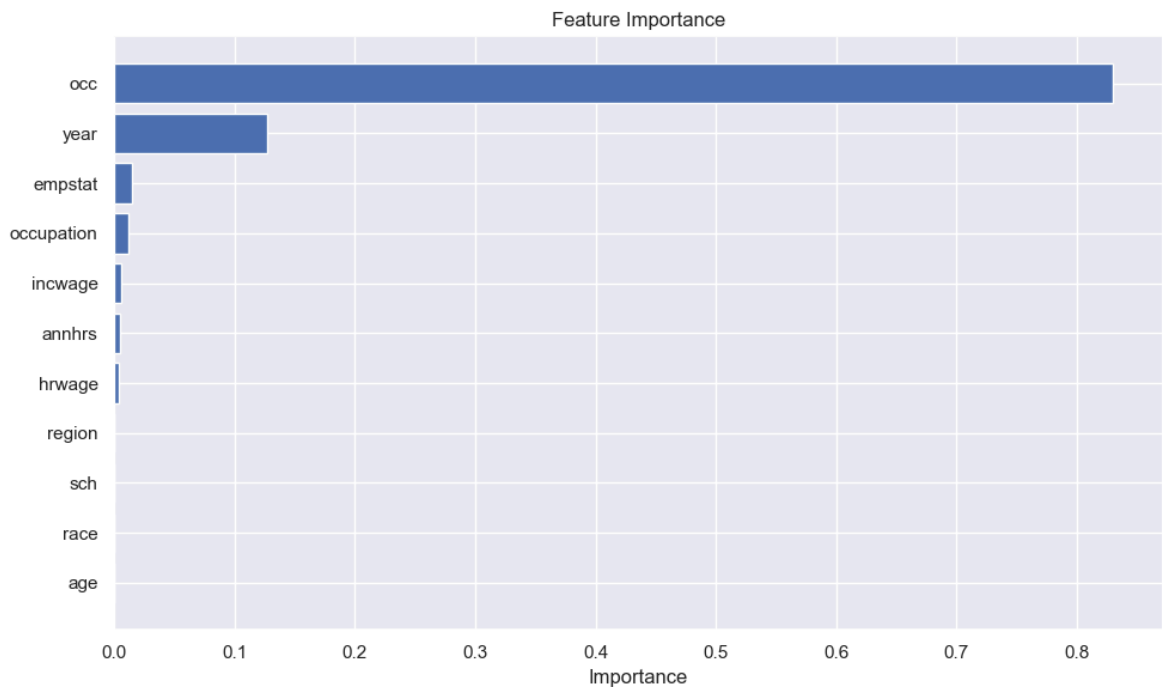
Но бустинг и дерево это, как говорится, две большие разницы. Дерево ничего не знает про лосс, оно учится по другим соображениям, которые на самом деле не так важны. И как оказывается, это влияние на лосс можно оценить.

Проблема предыдущего способа в том, что мы не учитываем, насколько сплит реально помогает в решении задачи минимизации функции потерь. Но зная лосс, а точнее градиент, это можно элементарно оценить

Алгоритм будет такой:

1. Возьмём базовый эстимейтор b_k (`sklearn.tree.DecisionTreeRegressor`)
2. Научимся доставать из него [путь](#) для конкретного объекта x_i
3. В этом пути для каждого узла R_j вытащим предсказание $b_k(R_j)(x_i)$ (это параметр `value`) и то, по какой фиче был сделан сплит
4. Для каждого предсказания запишем, на сколько уменьшился лосс (проще всего взять антиградиент, но можно взять критерий информативности из 2.4.1)

Повторить для всех деревьев, для всех объектов (или осмысленного их сабсэмпла)



Особо значительных изменений не было, но значения немного поменялись. Возможно это из-за небольшой глубины деревьев равной 5. Ниже проверю для глубины 10

```
In [17]: boosting_test = BoostingClassifier(
    verbose=False,
    base_model_params=dict(max_depth=5),
    n_estimators=50,
    random_state=69,
    eval_metric='valid_roc_auc',
    cat_features=cat_features,
    subsample=0.4,
    rsm = 0.5
)
boosting_test.fit(X_train, y_train, eval_set=(X_valid, y_valid))

print(f'Train ROC-AUC = {boosting_test.score(X_train, y_train):.4f}')
print(f'Valid ROC-AUC = {boosting_test.score(X_valid, y_valid):.4f}')
print(f'Test ROC-AUC = {boosting_test.score(X_test, y_test):.4f}')
```

```
Train ROC-AUC = 0.9543
Valid ROC-AUC = 0.7857
Test ROC-AUC = 0.7790
```

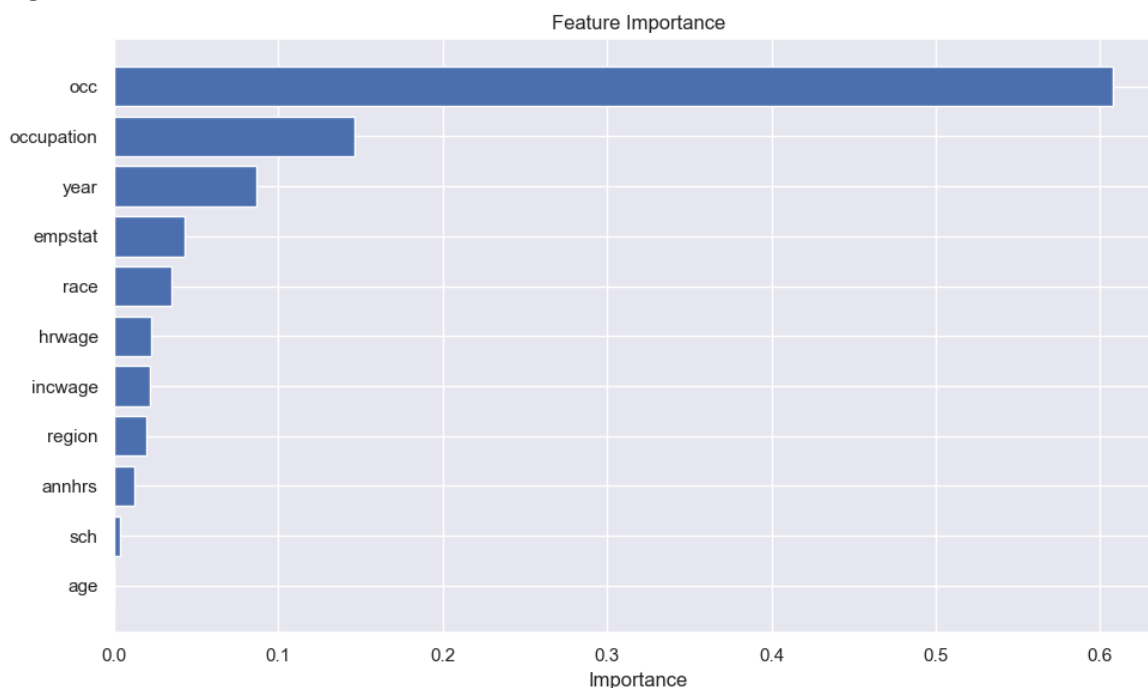
```
In [18]: importance_test = boosting_test.get_feature_importance(type='split')

order = np.argsort(importance_test[::-1])

print("Важность признаков:")
for i in order:
    print(f"{feature_names[i]}: {importance_test[i]:.4f}")

plt.figure(figsize=(10, 6))
plt.barh(range(len(importance_test)), importance_test[order], align='center')
plt.yticks(range(len(importance_test)), [feature_names[i] for i in order])
plt.xlabel('Importance')
plt.title('Feature Importance')
```


Важность признаков:
occ: 0.6079
occupation: 0.1466
year: 0.0865
empstat: 0.0433
race: 0.0347
hrwage: 0.0230
incwage: 0.0221
region: 0.0197
annhrs: 0.0122
sch: 0.0039
age: 0.0001



Благодаря бутстрапу и `rsm` получилось сильно увеличить важность разных признаков, а не делать важным только один. Видим что версия `split` теперь сильно отличается от `gain`.

Думаю, что из-за большей randomness в самом обучении модели у нас часто случается ситуация, что приходится делать сплиты по достаточно неинформативным признакам, поэтому `split` может завышать важность фичей из-за этого, поскольку он напрямую зависит от кол-ва таких сплитов. В случае `gain` мы смотрим именно вклад сплита в уменьшение функционала ошибки, поэтому к не особо значительным сплитам этот метод гораздо устойчивее, а значит на рандоме работает более надежно. Из-за вышеперечисленных моментов мы наблюдаем большую важность нижних фичей на графике, когда `gain` сильно занижает их на фоне реально полезных фичей, таких как `occ`, `year`, `occupation`.

2.9. DART (1 балл)

Ранее мы обсуждали, что каждый базовый алгоритм пытается исправить ошибки всего предыдущего ансамбля. Из-за этого возникает неравноправие — каждая новая модель вносит все меньший и меньший вклад. Уверенные обучатели

нейросетей знают, что это может привести к переобучению, если проводить аналогию с весами. Но выход есть, и выход этот [DART!](#)

В DL есть такой вид регуляризации, как dropout, когда некоторые узлы нейросети со случайным шансом отключаются. Можно попробовать обобщить этот подход на случай бустингов и случайно выкидывать деревья из композиции во время обучения

1. Выбираем, какие деревья убрать. При подсчете антиградиента берем не весь ансамбль, а только ту часть, что осталась
2. Домножаем выходы дерева на $1/k$, где k — число удаленных деревьев, чтобы учесть тот факт, что ансамбль был не весь. Тогда вклад отдельного дерева будет не настолько большим
3. Домножаем выходы выкинутых деревьев на $k/(k + 1)$, чтобы масштабы выходов примерно совпадали

Параметры, которые нужно добавить:

- `dart: bool = False` — DART бустинг или нет
- `dropout_rate: float = 0.05` — доля деревьев, которую выкидываем во время обучения DART

XGBoost

LightGBM

CatBoost

```
XGBClassifier(booster="dart")
```

```
LGBMClassifier(boosting_type="dart")
```

-

Попробуйте обучить DART на наших данных. Статью читать настоятельно рекомендуется, метод не самый простой

In []: # *??*??*??*??*??*??

2.10. Бустинг линейных моделей (0.5 балла)

Вопрос: что произойдет при ансамблировании линейных моделей? Имеет ли это смысл?

Ответ: На выходе у нас все еще должна получиться линейная модель, поскольку сумма линейных моделей все еще линейная модель. Возможно такой подход может ускорить обучение и сделать сходимость более стабильной и плавной, но может легко переобучиться, если это не контролировать.

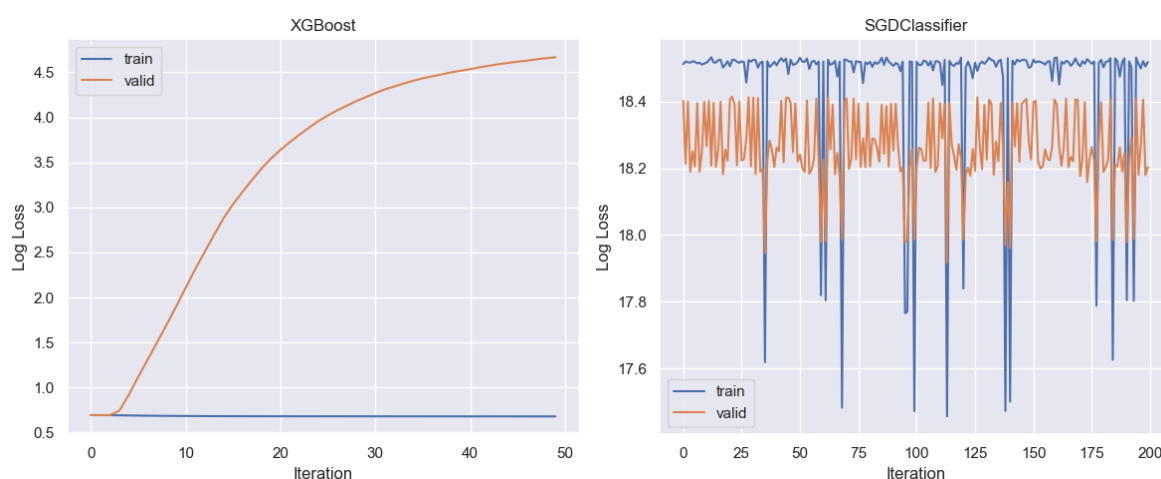
Давайте убедимся. Обучите бустинг, где в качестве базовой модели будет линейная регрессия. Обязательно сохраните историю функции потерь и зафиксируйте время обучения. Можно взять уже готовый из **XGBoost**, но будьте готовы разобраться с тем, как оттуда достать историю обучения

- время обучения
- сходимость (как быстро, как стабильно и прочее)
- графики функции потерь на трейне и на валидации

```
In [22]: plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(xgb_train_loss, label='train')
plt.plot(xgb_valid_loss, label='valid')
plt.xlabel('Iteration')
plt.ylabel('Log Loss')
plt.title('XGBoost')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(sgd_train_loss, label='train')
plt.plot(sgd_valid_loss, label='valid')
plt.xlabel('Iteration')
plt.ylabel('Log Loss')
plt.title('SGDClassifier')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

print(f"XGBoost time: {xgb_time}s, test loss: {xgb_test_loss}")
print(f"SGD time: {sgd_time}s, test loss: {sgd_test_loss}")
```



XGBoost time: 0.6456413269042969s, test loss: 3.6774630798630352

SGD time: 37.040467500686646s, test loss: 18.203418627420735

Вопрос: Какой вывод вы можете сделать?

Ответ: XGBoost обучается сильно быстрее, но в нем явно прослеживается переобучение с увеличением кол-ва итераций. XGBoost кажется гораздо более стабильным и успешным в плане сходимости. SGD что-то вообще не особо хочет сходиться видимо.



2.11. Потери потерь (0.5 балла)

Смотрите, какая существует штука:

Что же это за покемон?

- Если разметить объект легко, то есть сигмоида даёт результат, близкий к единице, то нам как бы максимально всё равно, какой у него вклад в лосс, множитель под гаммой будет примерно 0, интереснее же на сложных примерах обучаться. А если сложно, то вклад напротив максимальный, объект полностью участвует в расчёте лосса

- `loss: str = 'BCE'` — лосс, на который обучаем бустинг, по дефолту `'BCE'` - бинарная кросс-энтропия, aka логистическая функция потерь
- `focal_gamma: float = 2.0` — гиперпараметр для силы фокал лосса, чем-то похож на `class_weights`, но поинтереснее
- методы `loss_fn` и `grad_fn` для фокал лосса

```
loss_function="FocalLoss"
```

```
In [23]: # *~!*~!~!*~!*~!*~!  
boosting_bce = BoostingClassifier(  
    verbose=False,  
    base_model_params=dict(max_depth=5),  
    n_estimators=50,  
    random_state=69,  
    eval_metric='valid_roc_auc',  
    cat_features=cat_features,  
    loss='BCE'  
)  
boosting_bce.fit(X_train, y_train, eval_set=(X_valid, y_valid))  
  
boosting_focal = BoostingClassifier(
```

```

verbose=False,
base_model_params=dict(max_depth=5),
n_estimators=50,
random_state=69,
eval_metric='valid_roc_auc',
cat_features=cat_features,
loss='FocalLoss',
focal_gamma=3
)
boosting_focal.fit(X_train, y_train, eval_set=(X_valid, y_valid))

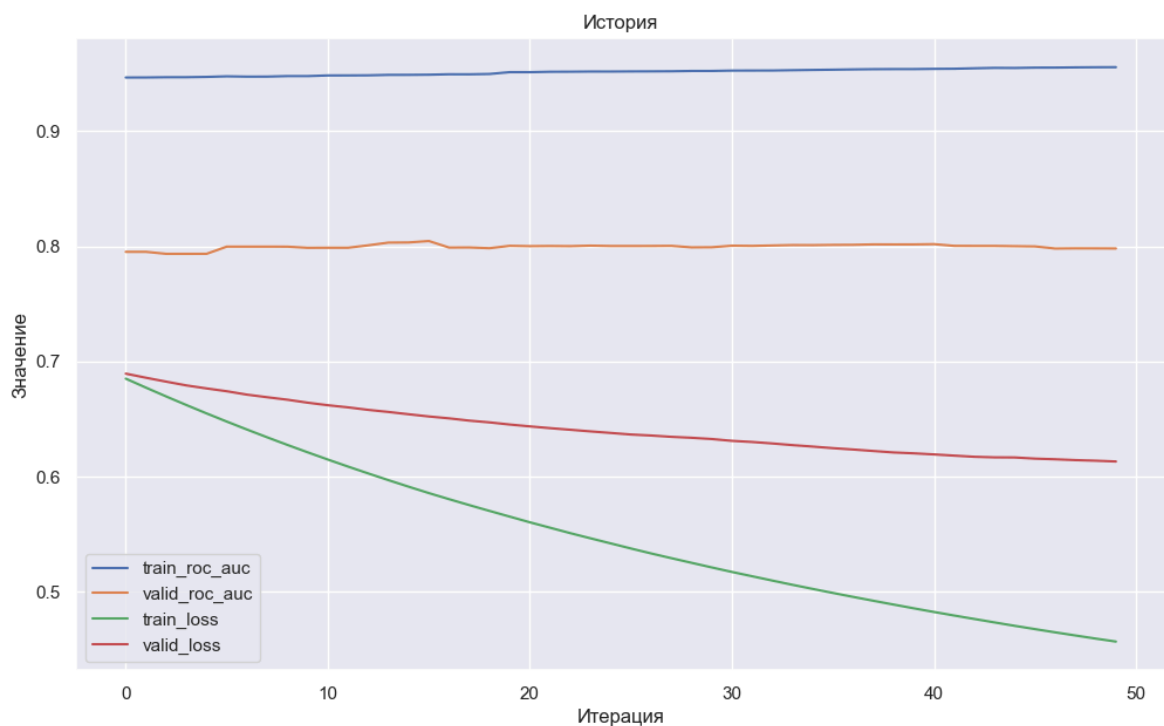
print(f'Train ROC-AUC = {boosting_bce.score(X_train, y_train):.4f}')
print(f'Valid ROC-AUC = {boosting_bce.score(X_valid, y_valid):.4f}')
print(f'Test ROC-AUC = {boosting_bce.score(X_test, y_test):.4f}')

print(f'Train ROC-AUC = {boosting_focal.score(X_train, y_train):.4f}')
print(f'Valid ROC-AUC = {boosting_focal.score(X_valid, y_valid):.4f}')
print(f'Test ROC-AUC = {boosting_focal.score(X_test, y_test):.4f}')

boosting_bce.plot_history(['train_roc_auc', 'valid_roc_auc', 'train_loss', 'valid_loss'])
boosting_focal.plot_history(['train_roc_auc', 'valid_roc_auc', 'train_loss', 'valid_loss'])

```

Train ROC-AUC = 0.9557
 Valid ROC-AUC = 0.7982
 Test ROC-AUC = 0.7913
 Train ROC-AUC = 0.9565
 Valid ROC-AUC = 0.8055
 Test ROC-AUC = 0.7987




```

)
study.optimize(objective, n_trials=30, show_progress_bar=True)

print("Лучшие параметры:")
for key, val in study.best_params.items():
    print(f"{key}: {val}")
print(f"Лучший ROC-AUC на валидации: {study.best_value:.4f}")

importance = optuna.importance.get_param_importances(study)
for param, imp in importance.items():
    print(f"{param}: {imp:.4f}")

```

[I 2026-05-24 22:46:19,190] A new study created in memory with name: no-name-d5dda55c-98b1-414a-9270-3903caf1f8f6

0%| | 0/30 [00:00<?, ?it/s]

[I 2026-05-24 22:47:18,670] Trial 0 finished with value: 0.6856435009548225 and parameters: {'learning_rate': 0.03186533139865215, 'n_estimators': 100, 'goss': False, 'max_depth': 9, 'bootstrap_type': 'Bernoulli', 'subsample': 0.836646190384922, 'rsm': 0.848914828500855, 'loss': 'FocalLoss', 'focal_gamma': 1.305505281574371}. Best is trial 0 with value: 0.6856435009548225.

[I 2026-05-24 22:48:06,328] Trial 1 finished with value: 0.8015657175412844 and parameters: {'learning_rate': 0.03465211959540217, 'n_estimators': 70, 'goss': False, 'max_depth': 5, 'bootstrap_type': None, 'rsm': 0.6465152743262423, 'loss': 'FocalLoss', 'focal_gamma': 1.933834802422608}. Best is trial 1 with value: 0.8015657175412844.

[I 2026-05-24 22:48:57,257] Trial 2 finished with value: 0.6596424649277058 and parameters: {'learning_rate': 0.14146094613383126, 'n_estimators': 40, 'goss': True, 'max_depth': 9, 'goss_k': 0.49507644574389476, 'subsample': 0.3123657485136616, 'rsm': 0.8402180788201157, 'loss': 'BCE'}. Best is trial 1 with value: 0.8015657175412844.

[I 2026-05-24 22:49:24,812] Trial 3 finished with value: 0.8085076258773953 and parameters: {'learning_rate': 0.15905791872190572, 'n_estimators': 40, 'goss': True, 'max_depth': 3, 'goss_k': 0.28308667841935736, 'subsample': 0.4223171634819929, 'rsm': 0.5967546057948767, 'loss': 'BCE'}. Best is trial 3 with value: 0.8085076258773953.

[I 2026-05-24 22:49:39,202] Trial 4 finished with value: 0.7755118298829982 and parameters: {'learning_rate': 0.013456145433630613, 'n_estimators': 10, 'goss': False, 'max_depth': 3, 'bootstrap_type': None, 'rsm': 0.9486542612320713, 'loss': 'BCE'}. Best is trial 3 with value: 0.8085076258773953.

[I 2026-05-24 22:51:13,027] Trial 5 finished with value: 0.8064549195471854 and parameters: {'learning_rate': 0.12793970984262384, 'n_estimators': 100, 'goss': False, 'max_depth': 5, 'bootstrap_type': None, 'rsm': 0.9913061842604174, 'loss': 'FocalLoss', 'focal_gamma': 2.247800082823302}. Best is trial 3 with value: 0.8085076258773953.

[I 2026-05-24 22:52:31,675] Trial 6 finished with value: 0.7622673759070109 and parameters: {'learning_rate': 0.45908260812850477, 'n_estimators': 40, 'goss': True, 'max_depth': 7, 'goss_k': 0.3324073963915827, 'subsample': 0.3366324530195859, 'rsm': 0.5290916043692718, 'loss': 'FocalLoss', 'focal_gamma': 1.0868081685780826}. Best is trial 3 with value: 0.8085076258773953.

[I 2026-05-24 22:54:02,053] Trial 7 finished with value: 0.7415816687634527 and parameters: {'learning_rate': 0.15117707335030459, 'n_estimators': 40, 'goss': False, 'max_depth': 7, 'bootstrap_type': 'Bernoulli', 'subsample': 0.5005039717950088, 'rsm': 0.5975363142677632, 'loss': 'FocalLoss', 'focal_gamma': 4.041591784850991}. Best is trial 3 with value: 0.8085076258773953.

[I 2026-05-24 22:54:50,935] Trial 8 finished with value: 0.8062724997116499 and parameters: {'learning_rate': 0.16644571854247214, 'n_estimators': 70, 'goss': False, 'max_depth': 5, 'bootstrap_type': 'Bernoulli', 'subsample': 0.8454300742229093, 'rsm': 0.7701757118120434, 'loss': 'FocalLoss', 'focal_gamma': 2.227251731020827}. Best is trial 3 with value: 0.8085076258773953.

[I 2026-05-24 22:55:10,660] Trial 9 finished with value: 0.7752997455611463 and parameters: {'learning_rate': 0.2885941203737404, 'n_estimators': 10, 'goss': True, 'max_depth': 5, 'goss_k': 0.456208157926672, 'subsample': 0.3314283956202325, 'rsm': 0.5338910336159652, 'loss': 'FocalLoss', 'focal_gamma': 2.680670940626077}. Best is trial 3 with value: 0.8085076258773953.

[I 2026-05-24 22:55:28,219] Trial 10 finished with value: 0.6654992082653272 and parameters: {'learning_rate': 0.05360141583410989, 'n_estimators': 10, 'goss': True, 'max_depth': 15, 'goss_k': 0.1098305409357597, 'subsample': 0.45296125822764044, 'rsm': 0.6797564457096861, 'loss': 'BCE'}. Best is trial 3 with value: 0.8085076258773953.

[I 2026-05-24 22:56:14,097] Trial 11 finished with value: 0.8085413143965822 and parameters: {'learning_rate': 0.0909929361685626, 'n_estimators': 100, 'goss': True, 'max_depth': 3, 'goss_k': 0.236976603001139, 'subsample': 0.13194842556530784, 'rsm': 0.9782790252131561, 'loss': 'BCE'}. Best is trial 11 with value: 0.8085413143965822.

[I 2026-05-24 22:57:08,323] Trial 12 finished with value: 0.6299089543118672 and

parameters: {'learning_rate': 0.07154491238524449, 'n_estimators': 70, 'goss': True, 'max_depth': 13, 'goss_k': 0.24111882936841247, 'subsample': 0.10165991730134452, 'rsm': 0.7317908133341627, 'loss': 'BCE'}. Best is trial 11 with value: 0.8085413143965822.

[I 2026-05-24 22:58:38,216] Trial 13 finished with value: 0.8092318056160113 and parameters: {'learning_rate': 0.07654038596677964, 'n_estimators': 100, 'goss': True, 'max_depth': 3, 'goss_k': 0.2625180665144752, 'subsample': 0.12253774192154901, 'rsm': 0.899775984473094, 'loss': 'BCE'}. Best is trial 13 with value: 0.8092318056160113.

[I 2026-05-24 23:00:30,786] Trial 14 finished with value: 0.8068598933610329 and parameters: {'learning_rate': 0.07056459591196527, 'n_estimators': 100, 'goss': True, 'max_depth': 3, 'goss_k': 0.1914161452884337, 'subsample': 0.10000833920820562, 'rsm': 0.9042596076605816, 'loss': 'BCE'}. Best is trial 13 with value: 0.8092318056160113.

[I 2026-05-24 23:03:33,180] Trial 15 finished with value: 0.6521879150757843 and parameters: {'learning_rate': 0.025043467529453196, 'n_estimators': 100, 'goss': True, 'max_depth': 11, 'goss_k': 0.36024740524559706, 'subsample': 0.17997987020751505, 'rsm': 0.9014962289027479, 'loss': 'BCE'}. Best is trial 13 with value: 0.8092318056160113.

[I 2026-05-24 23:04:40,273] Trial 16 finished with value: 0.6399087642528507 and parameters: {'learning_rate': 0.07305887168360924, 'n_estimators': 70, 'goss': True, 'max_depth': 7, 'goss_k': 0.19349924141559063, 'subsample': 0.18871005557409182, 'rsm': 0.9989856204946177, 'loss': 'BCE'}. Best is trial 13 with value: 0.8092318056160113.

[I 2026-05-24 23:05:20,953] Trial 17 finished with value: 0.8068543792776086 and parameters: {'learning_rate': 0.015795485007048825, 'n_estimators': 100, 'goss': True, 'max_depth': 3, 'goss_k': 0.3947125081556806, 'subsample': 0.18272317225642054, 'rsm': 0.8415306281128531, 'loss': 'BCE'}. Best is trial 13 with value: 0.8092318056160113.

[I 2026-05-24 23:07:25,886] Trial 18 finished with value: 0.6729035962996891 and parameters: {'learning_rate': 0.0484516510952025, 'n_estimators': 100, 'goss': True, 'max_depth': 11, 'goss_k': 0.2560882886591688, 'subsample': 0.23608742498701218, 'rsm': 0.9244848563679445, 'loss': 'BCE'}. Best is trial 13 with value: 0.8092318056160113.

[I 2026-05-24 23:07:58,612] Trial 19 finished with value: 0.7538677561475025 and parameters: {'learning_rate': 0.0944083045334445, 'n_estimators': 70, 'goss': True, 'max_depth': 7, 'goss_k': 0.1545326699992331, 'subsample': 0.14330215288796025, 'rsm': 0.7778065141266345, 'loss': 'BCE'}. Best is trial 13 with value: 0.8092318056160113.

[I 2026-05-24 23:09:43,794] Trial 20 finished with value: 0.8096050263904985 and parameters: {'learning_rate': 0.2946520098634052, 'n_estimators': 100, 'goss': True, 'max_depth': 3, 'goss_k': 0.22231965028033793, 'subsample': 0.24954031254310016, 'rsm': 0.9557487705401161, 'loss': 'BCE'}. Best is trial 20 with value: 0.8096050263904985.

[I 2026-05-24 23:11:21,860] Trial 21 finished with value: 0.8086043784936255 and parameters: {'learning_rate': 0.33220667397640674, 'n_estimators': 100, 'goss': True, 'max_depth': 3, 'goss_k': 0.22337704021700483, 'subsample': 0.2552422713225123, 'rsm': 0.9420733498082147, 'loss': 'BCE'}. Best is trial 20 with value: 0.8096050263904985.

[I 2026-05-24 23:11:55,423] Trial 22 finished with value: 0.8076509116236854 and parameters: {'learning_rate': 0.27518575490326314, 'n_estimators': 100, 'goss': True, 'max_depth': 5, 'goss_k': 0.19682956960631925, 'subsample': 0.2598155493061351, 'rsm': 0.8628329182735567, 'loss': 'BCE'}. Best is trial 20 with value: 0.8096050263904985.

[I 2026-05-24 23:12:58,680] Trial 23 finished with value: 0.8048483837223751 and parameters: {'learning_rate': 0.4411107858180399, 'n_estimators': 70, 'goss': True, 'max_depth': 3, 'goss_k': 0.30167742194162617, 'subsample': 0.27036048754643727, 'rsm': 0.9525835703298446, 'loss': 'BCE'}. Best is trial 20 with value: 0.8096050263904985.

[I 2026-05-24 23:13:37,667] Trial 24 finished with value: 0.8080351805610717 and

parameters: {'learning_rate': 0.26774302956688567, 'n_estimators': 100, 'goss': True, 'max_depth': 5, 'goss_k': 0.15422760548560022, 'subsample': 0.22578825461946014, 'rsm': 0.8852426983689234, 'loss': 'BCE'}. Best is trial 20 with value: 0.8096050263904985.

[I 2026-05-24 23:16:01,777] Trial 25 finished with value: 0.8129352748985786 and parameters: {'learning_rate': 0.23698128293894047, 'n_estimators': 100, 'goss': True, 'max_depth': 3, 'goss_k': 0.28476333441562, 'subsample': 0.36878918401157396, 'rsm': 0.8115233750665507, 'loss': 'BCE'}. Best is trial 25 with value: 0.8129352748985786.

[I 2026-05-24 23:16:44,307] Trial 26 finished with value: 0.695482730885002 and parameters: {'learning_rate': 0.20360605218865366, 'n_estimators': 70, 'goss': True, 'max_depth': 9, 'goss_k': 0.2907407819692094, 'subsample': 0.40635968496107433, 'rsm': 0.7996341889831127, 'loss': 'BCE'}. Best is trial 25 with value: 0.8129352748985786.

[I 2026-05-24 23:17:21,706] Trial 27 finished with value: 0.8086401665981601 and parameters: {'learning_rate': 0.22936443148057997, 'n_estimators': 100, 'goss': True, 'max_depth': 5, 'goss_k': 0.3443658866360306, 'subsample': 0.3749048419285499, 'rsm': 0.7257000838858151, 'loss': 'BCE'}. Best is trial 25 with value: 0.8129352748985786.

[I 2026-05-24 23:18:36,139] Trial 28 finished with value: 0.8165504519630167 and parameters: {'learning_rate': 0.3674567835275852, 'n_estimators': 70, 'goss': True, 'max_depth': 3, 'goss_k': 0.3948599196890892, 'subsample': 0.36695981575496733, 'rsm': 0.8011561533921491, 'loss': 'BCE'}. Best is trial 28 with value: 0.8165504519630167.

[I 2026-05-24 23:19:44,834] Trial 29 finished with value: 0.6971186388400739 and parameters: {'learning_rate': 0.4745341466406585, 'n_estimators': 70, 'goss': False, 'max_depth': 11, 'bootstrap_type': 'Bayesian', 'bagging_temperature': 2.9896946514586373, 'rsm': 0.8164168219386684, 'loss': 'BCE'}. Best is trial 28 with value: 0.8165504519630167.

Лучшие параметры:

learning_rate: 0.3674567835275852

n_estimators: 70

goss: True

max_depth: 3

goss_k: 0.3948599196890892

subsample: 0.36695981575496733

rsm: 0.8011561533921491

loss: BCE

Лучший ROC-AUC на валидации: 0.8166

max_depth: 0.8441

rsm: 0.0757

learning_rate: 0.0646

n_estimators: 0.0058

goss: 0.0051

loss: 0.0046

Важность:

max_depth: 0.8441

rsm: 0.0757

learning_rate: 0.0646

n_estimators: 0.0058

goss: 0.0051

loss: 0.0046

Выбейте ROC-AUC хотя бы 0.8, и можете двигаться дальше

Выбил 0.8166, если нужно было выбить 0,8 на тесте, то можно посмотреть на модели выше, которые я уже делал, там 0.8 уже давно на test выбита.

3.2. Готовые реализации (0.5 балла)

Библиотек для бустингов существует много, но активно используются лишь три. Вам предстоит попробовать их в деле, посмотреть на их уникальные фишки и понять, что вам нравится больше всего

Возьмите любую из популярных библиотек: **XGBoost**, **LightGBM** или **CatBoost**. Ваша задача — взять одну из них, подобрать те же оптимальные параметры, что и выше, а также вставить [прунер](#).

Идея прунера — не обучать бустинг до конца, если по истории валидации видно, что другие бустинги на этом этапе статистически лучше, для этого мы и делали `partial_fit`, но в готовые реализации его можно вставить при помощи `CallBack`, для этого придётся немножко погуглить. Это во-первых, заметно ускоряет процесс перебора гиперпараметров, во-вторых неявно регуляризует модельку

Цель — увидеть в логах оптуны `Trial.PRUNED`, это знак того, что прунер работает. Если вы этого не видите, вероятно, вы вставили его неправильно.

[illegible]

```

    preds = model.predict_proba(X_valid[:, :-1])[:, 1]
    return roc_auc_score(y_valid_01, preds)

study = optuna.create_study(
    direction="maximize",
    pruner=MedianPruner(n_warmup_steps=5, n_min_trials=5),
    sampler=optuna.samplers.TPESampler(seed=69)
)
study.optimize(objective_new, n_trials=30, show_progress_bar=True)

for k, v in study.best_params.items():
    print(f"{k}: {v}")
print(f"Лучший ROC-AUC: {study.best_value:.4f}")

```

[I 2026-05-24 23:43:04,120] A new study created in memory with name: no-name-e698fba6-d6b6-404c-997a-5deb8d61f157

0%| | 0/30 [00:00<?, ?it/s]

[I 2026-05-24 23:43:06,324] Trial 0 finished with value: 0.7667951720464498 and parameters: {'learning_rate': 0.02739024603777705, 'max_depth': 13, 'num_leaves': 54, 'reg_alpha': 7.8940925643089255, 'reg_lambda': 5.613489757705725, 'subsample': 0.6267944771100666, 'colsample_bytree': 0.5524885418425545}. Best is trial 0 with value: 0.7667951720464498.

[I 2026-05-24 23:43:08,904] Trial 1 finished with value: 0.7651150872987178 and parameters: {'learning_rate': 0.012199824569279061, 'max_depth': 11, 'num_leaves': 94, 'reg_alpha': 7.327832059516539, 'reg_lambda': 7.878740584977104, 'subsample': 0.5381881601967964, 'colsample_bytree': 0.6588402983099821}. Best is trial 0 with value: 0.7667951720464498.

[I 2026-05-24 23:43:10,674] Trial 2 finished with value: 0.8003811176976632 and parameters: {'learning_rate': 0.12433151710093181, 'max_depth': 7, 'num_leaves': 68, 'reg_alpha': 1.810857177346843, 'reg_lambda': 3.2350440180910067, 'subsample': 0.7274530779834941, 'colsample_bytree': 0.7648152577408763}. Best is trial 2 with value: 0.8003811176976632.

[I 2026-05-24 23:43:11,965] Trial 3 finished with value: 0.725448950025431 and parameters: {'learning_rate': 0.027092036478460972, 'max_depth': 5, 'num_leaves': 92, 'reg_alpha': 2.3345870060565197, 'reg_lambda': 6.772553697317375, 'subsample': 0.7161662487834132, 'colsample_bytree': 0.6938331426328919}. Best is trial 2 with value: 0.8003811176976632.

[I 2026-05-24 23:43:15,272] Trial 4 finished with value: 0.7899135946092368 and parameters: {'learning_rate': 0.03129642434061462, 'max_depth': 9, 'num_leaves': 127, 'reg_alpha': 5.309143712841541, 'reg_lambda': 6.8043615764023135, 'subsample': 0.6955673666207349, 'colsample_bytree': 0.5582780977258559}. Best is trial 2 with value: 0.8003811176976632.

[I 2026-05-24 23:43:17,158] Trial 5 finished with value: 0.7928895158528844 and parameters: {'learning_rate': 0.1108302090231576, 'max_depth': 7, 'num_leaves': 66, 'reg_alpha': 3.3195253580190287, 'reg_lambda': 0.9371524331626679, 'subsample': 0.7288583480241967, 'colsample_bytree': 0.9028964543524911}. Best is trial 2 with value: 0.8003811176976632.

[I 2026-05-24 23:43:17,365] Trial 6 pruned. Trial was pruned at iteration 6.

[I 2026-05-24 23:43:17,576] Trial 7 pruned. Trial was pruned at iteration 6.

[I 2026-05-24 23:43:20,133] Trial 8 finished with value: 0.8027529347990201 and parameters: {'learning_rate': 0.09171796188767664, 'max_depth': 13, 'num_leaves': 65, 'reg_alpha': 5.328021083690589, 'reg_lambda': 1.629177546747762, 'subsample': 0.77757928285676, 'colsample_bytree': 0.563392573751139}. Best is trial 8 with value: 0.8027529347990201.

[I 2026-05-24 23:43:22,477] Trial 9 finished with value: 0.8078379608897708 and parameters: {'learning_rate': 0.24943971202975534, 'max_depth': 15, 'num_leaves': 49, 'reg_alpha': 7.696255696817746, 'reg_lambda': 3.1195002070582545, 'subsample': 0.9890877520019175, 'colsample_bytree': 0.640639077394517}. Best is trial 9 with value: 0.8078379608897708.

[I 2026-05-24 23:43:22,698] Trial 10 pruned. Trial was pruned at iteration 7.

[I 2026-05-24 23:43:25,278] Trial 11 finished with value: 0.8092721804009184 and parameters: {'learning_rate': 0.278422885981822, 'max_depth': 15, 'num_leaves': 44, 'reg_alpha': 6.054701892550279, 'reg_lambda': 2.417530068992362, 'subsample': 0.9153316699306266, 'colsample_bytree': 0.5023821985361427}. Best is trial 11 with value: 0.8092721804009184.

[I 2026-05-24 23:43:27,489] Trial 12 finished with value: 0.8091834930884036 and parameters: {'learning_rate': 0.2760971641314985, 'max_depth': 15, 'num_leaves': 42, 'reg_alpha': 7.135468052830665, 'reg_lambda': 3.2458335153416273, 'subsample': 0.9736634309969241, 'colsample_bytree': 0.5099786563008026}. Best is trial 11 with value: 0.8092721804009184.

[I 2026-05-24 23:43:27,721] Trial 13 pruned. Trial was pruned at iteration 6.

[I 2026-05-24 23:43:28,006] Trial 14 pruned. Trial was pruned at iteration 6.

[I 2026-05-24 23:43:28,278] Trial 15 pruned. Trial was pruned at iteration 6.

[I 2026-05-24 23:43:28,620] Trial 16 pruned. Trial was pruned at iteration 18.

[I 2026-05-24 23:43:30,988] Trial 17 finished with value: 0.8082673581560338 and parameters: {'learning_rate': 0.20732002829549817, 'max_depth': 15, 'num_leaves': 51, 'reg_alpha': 4.616130519215385, 'reg_lambda': 1.2818346801263114, 'subsample':

e': 0.9363526784275329, 'colsample_bytree': 0.6096489026863144}. Best is trial 11 with value: 0.8092721804009184.
[I 2026-05-24 23:43:31,277] Trial 18 pruned. Trial was pruned at iteration 6.
[I 2026-05-24 23:43:34,532] Trial 19 finished with value: 0.805101102088277 and parameters: {'learning_rate': 0.2995757428313599, 'max_depth': 11, 'num_leaves': 113, 'reg_alpha': 5.776478895094796, 'reg_lambda': 5.934862010815694, 'subsample': 0.9392588014338461, 'colsample_bytree': 0.5038841291335496}. Best is trial 11 with value: 0.8092721804009184.
[I 2026-05-24 23:43:34,838] Trial 20 pruned. Trial was pruned at iteration 6.
[I 2026-05-24 23:43:37,072] Trial 21 finished with value: 0.813902727282348 and parameters: {'learning_rate': 0.21460658520740827, 'max_depth': 15, 'num_leaves': 46, 'reg_alpha': 4.292619366185002, 'reg_lambda': 1.6837030394266888, 'subsample': 0.9380418055358247, 'colsample_bytree': 0.5991944848501576}. Best is trial 21 with value: 0.813902727282348.
[I 2026-05-24 23:43:37,329] Trial 22 pruned. Trial was pruned at iteration 6.
[I 2026-05-24 23:43:37,596] Trial 23 pruned. Trial was pruned at iteration 6.
[I 2026-05-24 23:43:37,829] Trial 24 pruned. Trial was pruned at iteration 5.
[I 2026-05-24 23:43:38,097] Trial 25 pruned. Trial was pruned at iteration 6.
[I 2026-05-24 23:43:40,587] Trial 26 finished with value: 0.8124264960681931 and parameters: {'learning_rate': 0.24674382075591914, 'max_depth': 13, 'num_leaves': 59, 'reg_alpha': 8.883626258564878, 'reg_lambda': 3.8338713529168107, 'subsample': 0.783449848963045, 'colsample_bytree': 0.5836355006624356}. Best is trial 21 with value: 0.813902727282348.
[I 2026-05-24 23:43:40,918] Trial 27 pruned. Trial was pruned at iteration 5.
[I 2026-05-24 23:43:42,289] Trial 28 pruned. Trial was pruned at iteration 97.
[I 2026-05-24 23:43:42,553] Trial 29 pruned. Trial was pruned at iteration 6.
learning_rate: 0.21460658520740827
max_depth: 15
num_leaves: 46
reg_alpha: 4.292619366185002
reg_lambda: 1.6837030394266888
subsample: 0.9380418055358247
colsample_bytree: 0.5991944848501576
Лучший ROC-AUC: 0.8139

Вопрос: Какая архитектура вам нравится больше всего и почему?

Ответ: Сначала использовал XGBoost, но код все никак не хотел работать и гпт тоже не смог помочь пофиксить багу, поэтому решил перейти на LightGBM. Как я понял, если сравнивать XGBoost с LightGBM, то последний более быстрый и легкий в плане потребляемой памяти, а первый позволяет строить более сложные деревья.

3.3. XGBoost и велосипеды (0.25 баллов)

XGBoost известен тем, что в нем впервые повилась регуляризация. Ручками ее написать сложно, и вы даже могли попробовать пунктом ранее, но зачем? Она уже написана, вообще-то. Вместо того, чтобы заниматься ерундой, можем попробовать импортировать и посмотреть, насколько она полезна в лучшем своём представлении.

Добавьте оба вида регуляризации, что там есть, как параметры в оптуну, и прогоните процесс оптимизации только на них.

XGBoost**LightGBM****CatBoost**`alpha , lambda    lambda_l1 , lambda_l2    - , l2_leaf_reg`

In []: # *~*~*~*~*~*~*

Вопрос: Насколько регуляризация вообще эффективна?**Ответ:**

❖ 3.4. **LightGBM**, лес vs бустинг (0.25 балла)

LightGBM велик по многим причинам. В нём самое большое количество гиперпараметров из всех библиотек. А ещё он содержит в себе реализацию своего злейшего врага — случайный лес, это очень по-философски.

Посмотрите, какие есть параметры у него, обучите так сильно, как только сможете (не забудьте про свойства случайного леса к переобучению, про категориальные энкодеры и прочее).

XGBoost**LightGBM****CatBoost**`num_parallel_tree    boosting_type="rf"    -`

In [26]: # *~*~*~*~*~*~*

Вопрос: Кто победит: бустинг или лес? и почему?**Ответ:**

❖ 3.5. Еще пара красивых слов про **CatBoost** (0.25 балла)

CatBoost славится ещё и кучей разных подходов к обучению деревьев, в частности он умеет мёрджить несколько моделей в одно. Это не стекинг, и даже не беггинг, а совершенно другой уровень. Поскольку бустинги это уже некоторая сумма деревьев, ничего не мешает брать и складывать уже сами бустинги между собой.

Попробуйте разбить выборку на фолды и обучить по катбусту на каждом из них, а затем их склеить. Тюнить их не надо, он и так неплох, в этом его сила и мудрость.

XGBoost **LightGBM****CatBoost**`-    -    sum_models , to_classifier`

Оцените, насколько это круто по 10-балльной

In []: # *??*??*??*??*??

3.6. Бустинг и вероятности (0.5 балла)

С качеством мы более-менее разобрались, но таки что насчет вероятностей? Постройте калибровочную кривую для вашей лучшей модели бустинга. Насколько хорошо бустинг оценивает вероятности?

In []: # *??*??*??*??*??

Постройте также калибровочную кривую для логистической регрессии, сравните их между собой. Проанализируйте полученные результаты.

In []: # *??*??*??*??*??

Конечно же, бустинг великолепен, как он есть, но можно сделать его еще лучше, если его откалибровать

1. Посмотрите на то, как это делать из библиотеки `sklearn`. Обратите внимание, передаете вы в него обученную модель или нет, а также на чем вы калибруетесь. Иначе есть риск что-то поломать
2. Откалибруйте вашу реализацию бустинга. Для этого за вас к нему был замешан `ClassifierMixin`
3. Покажите, как изменилась мера откалиброванности, для этого есть

$$\text{LogLoss} = -\frac{1}{N} \sum_i \sum_{k \in 0.1} \log p_k[y_i = k]; \quad \text{BrierScore} = \frac{1}{N} \sum_i (p_i - y_i)^2;$$

4. Нарисуйте новую калибровочную кривую

In []: # *??*??*??*??*??

Вопрос: Как вы можете прокомментировать то, что получилось?

Ответ:

Музыкальный бонус. (0.5 балла) 🎵

В честь окончания весны исполните песню, которая больше всего ассоциируется у вас с МО-1, будь то Ольга Бузова, Валентин Стрыкало, Игорь Вихорьков, Глеб Смирнов или любой другой заслуженный артист. Одни, в бойз или гёрлз-бенде, вокально или инструментально (например, вашему вниманию представляется раритетнейшая [аранжировка](#) на незабвенную классику за авторством Михаила Шелега), проявите фантазию.

